CHAPTER 3

Informed Search: Greedy Best-First Search and A*

Chapter at a glance

Before you start: Trace BFS, DFS and UCS (Chapter 2) and explain what a priority queue does.
By the end, you can: define a heuristic and compute g, h and f for a state; trace Greedy Best-First Search and A*; decide whether a heuristic is admissible and why that matters; and read the effort-versus-quality trade-off in Lab 2's tables.

UCS found the shortest Kuala Lumpur–Keflavík route but expanded 1,851 airports without geographical guidance. Lab 2 adds a **heuristic** — an estimate of remaining distance — and measures its effect on search effort and route quality.

3.1 Three quantities on one path

Every informed method reasons about a frontier state n using up to three numbers:

Quantity	Meaning	Used by
$g(n)$	Exact cost of the cheapest known path from the start to n	UCS, A*
$h(n)$	Estimated cost from n to the goal — the heuristic	GBFS, A*
$f(n) = g(n) + h(n)$	Estimated total cost of a solution through n	A*

Figure 3.1 separates known cost from estimated remaining cost: UCS uses the former, GBFS the latter, and A* both.

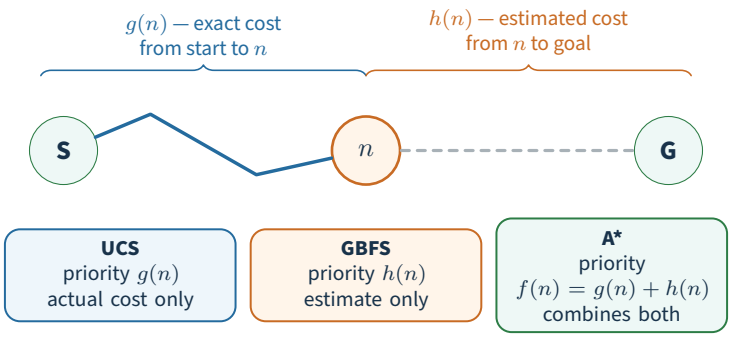


Figure 3.1. $f(n) = g(n) + h(n)$ combines known cost with estimated remaining cost. UCS prioritises g, GBFS h, and A* their sum.

Key idea

GBFS asks which frontier state *looks* closest to the goal. A* also accounts for the cost already paid and, with a suitable heuristic, guarantees the cheapest path.

3.2 The Haversine heuristic

A heuristic should be cheap to compute and use the same units as path cost. For kilometre-based routes, use the great-circle distance to the goal, whether or not a direct flight exists. The **Haversine formula** computes it from latitude–longitude pairs:

$$h(n) = 2R \arcsin\sqrt{\sin^2\frac{\Delta\varphi}{2} + \cos\varphi_n \cos\varphi_{\text{goal}} \sin^2\frac{\Delta\lambda}{2}}$$

with $R = 6371$ km. The same formula gives every edge weight in the graph, so h and g are directly comparable. In Lab 2 the heuristic is a small function that fixes the goal and returns a function of the airport:

```
def heuristic_to(goal, weight=1.0):
    def h(airport):
        return weight * haversine_km(coords[airport]["latitude"], coords[airport]["longitude"],
                                     coords[goal]["latitude"], coords[goal]["longitude"])
    return h
```

`utils/core.py` accepts any state-to-number callable, so changing the heuristic requires no changes to the search code.

3.3 Greedy Best-First Search

Greedy Best-First Search (GBFS) prioritises $h(n)$ alone, ignoring the cost already paid. It finds routes in two or three expansions on the airline graph, but need not find cheap ones.

Worked example

GBFS expands two airports for `KUL -> LHR -> KEF`: 12,500 km, 9 % above the optimum. London looks close to Iceland, but the first leg alone exceeds the optimal route through Phuket, St Petersburg and Helsinki. Its `NRT→CPT` route is 6.7 % too long; its `SIN→EZE` route happens to be optimal.

Common mistake

Because GBFS ignores g, a small h can conceal an expensive path. Route quality depends on the graph around the goal.

3.4 A* search

A* prioritises $f(n) = g(n) + h(n)$, balancing cost so far against estimated cost ahead. With a consistent h, this closed-set implementation returns the cheapest path, usually with far fewer expansions than UCS.

Worked example

Suppose frontier state X has $(g, h) = (5, 3)$ and Y has $(1, 5)$. GBFS chooses X because $3 < 5$. A* chooses Y because $f(Y) = 6 < f(X) = 8$: the cheaper path so far is worth the extra estimated distance.

Implementation is one line away from UCS. In `utils/core.py` the three algorithms share `best_first`; only the priority differs:

```
def priority(state, g):
    if mode == "ucs":
        return g
    if mode == "gbfs":
        return h(state)
    return g + h(state)            # astar
```

3.4.1 Admissibility and consistency

The optimality guarantee rests on one condition. A heuristic is **admissible** when it never overestimates the cheapest remaining cost:

$$h(n) \;\le\; h^\star(n) \quad \text{for every state } n,$$

where $h^\star(n)$ is the true cost of the cheapest path from n to the goal. A stronger and more convenient condition is **consistency**: for every edge $n \to n'$ of cost c, $h(n) \le c + h(n')$. Consistency implies admissibility and guarantees that the first time A* pops a state, it has found the cheapest path to it — which is what allows the closed-set implementation in `utils` to skip re-expansions.

Haversine is consistent by the triangle inequality on a sphere. Lab 2 also runs Dijkstra backwards from each goal to obtain $h^\star$, samples ten airports across the range, and checks $h \le h^\star$. In Figure 3.2, every sampled point must lie on or below the diagonal.

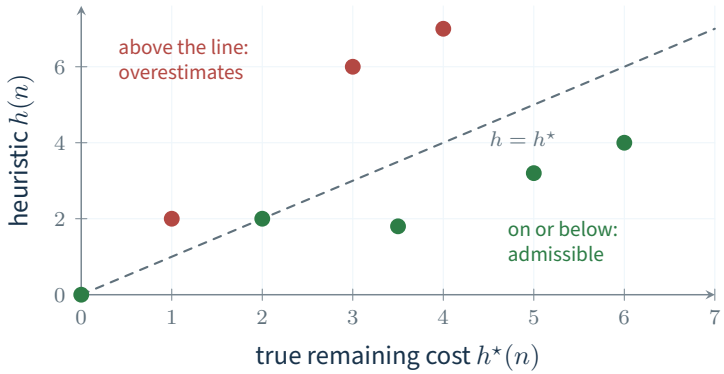


Figure 3.2. A heuristic plotted against the true remaining cost. Admissibility requires every point to lie on or below the dashed line. The red points overestimate and void the A* guarantee; the Haversine heuristic in Lab 2 produces only green points, with slack of 0 to 23,000 km.

Conceptual intuition

An inadmissible heuristic makes A* behave like GBFS: it trusts the estimate too much and commits to a branch before the cost evidence justifies it. Admissibility keeps the g term in charge whenever the two disagree.

3.4.2 Weighted A*: trading the guarantee for speed

Weighted A* uses $g + w\,h$ with $w > 1$, trading the optimality guarantee for more goal-directed search. At $w = 1.5$, Lab 2 uses two, three and four expansions, with routes 7.1 %, 0 % and 0.8 % above optimum. Inflating h can discourage useful detours away from the goal.

3.5 UCS, GBFS and A* side by side

Method	Priority	Optimal?	Behaviour in Lab 2
UCS	$g(n)$	Yes	Exact but blind: 1,851–3,042 expansions per query
GBFS	$h(n)$	No	2–3 expansions; routes up to 9 % too long
A*	$g(n) + h(n)$	Yes, if h consistent	Same cost as UCS with 96 % fewer expansions
Weighted A*	$g(n) + 1.5\,h(n)$	No	2–4 expansions; routes up to 7 % too long

Figure 3.3 visualises these expansion patterns on a shared grid.

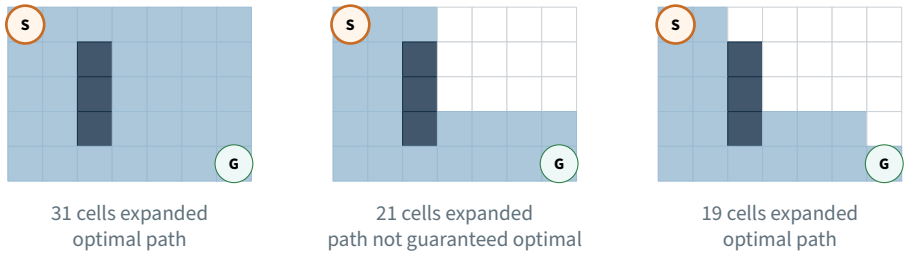


Figure 3.3. Expansion on an identical grid: UCS spreads widely, GBFS heads towards the goal but enters a dead end, and A* finds the optimal path through a narrower corridor.

Key idea

A* combines low effort with optimality here because Haversine is consistent. UCS explores without guidance; GBFS reduces effort but gives up the guarantee.

3.6 Reading the lab's evidence

`metrics.csv` records cost, hops, expansions, peak frontier, runtime and the **optimality gap**: percentage cost above UCS. A* has zero gap on every query; positive GBFS or weighted A* gaps quantify the cost of speed. `heuristic_check.csv` compares h with h^* for thirty samples. Any failed check would invalidate the admissibility claim.